

AUTONOMOUS AGENTS · SERIES TWO

The Scholar Series

Lesson 1 — The Agentic Loop.

In series one, you called an agent. Here, you set one loose and watch it work.

New series · we build in LangGraph and CrewAI side by side

A different kind of agent. The ShopHub series taught the *reactive* agent: it waited for a message and responded. This series teaches the *autonomous* agent: you give it one goal and it works on its own, over many steps, until the job is done. Meet Scholar, an agent that researches a topic and writes a report — and along the way, learn to build the same ideas in two different frameworks and judge which fits best.

What this lesson covers

- 1 **The scene** — why a one-shot answer fails a real research goal
- 2 **The concept** — the think-act-observe loop that makes an agent autonomous
- 3 **The architecture** — a loop with an exit, not a straight line
- 4 **Build it in LangGraph** — the loop as an explicit graph with a cycle
- 5 **Build it in CrewAI** — the same loop, hidden inside an agent
- 6 **Compare** — the pros and cons of each, side by side
- 7 **Run it + the new crack** — and why a loop without a plan wanders

PART 1

The scene: a question too big for one breath

Imagine you ask an ordinary chatbot, the kind we built in series one, a genuinely big research question: "Write me a briefing on the current state of solid-state batteries." Watch what comes back.

You: Write me a briefing on the current state of solid-state batteries.

One-shot bot: Solid-state batteries replace the liquid electrolyte with a solid one, offering higher energy density and safety. Several companies are working on them. They may reach commercial production in the coming years.

It's not *wrong*, but it's thin, generic, and a little stale. A real briefing would compare specific companies, cite recent milestones, weigh the manufacturing hurdles, and give a grounded timeline. The chatbot answered in a single pass from frozen memory, because that is all a one-shot agent can do: it receives a prompt and produces one response. It cannot go and *look*, notice what's missing, look again, and build up a real answer.

THE FAILURE, PRECISELY

A real research goal can't be satisfied in one pass. It requires *working at it*: searching, reading, realising a gap, searching again, and assembling findings over many steps. A one-shot agent has no mechanism to take a second step based on what the first step revealed. What's missing is a *loop*.

PART 2

The concept: think, act, observe, repeat

Here is the idea that turns a chatbot into an autonomous agent, and it is the foundation of this entire series. An autonomous agent runs in a loop with four beats. It *thinks* about what it needs next. It *acts*, taking a concrete step like running a search. It *observes* the result of that action. And then it *decides* whether the goal is met — if not, it loops back and thinks again. This pattern is often called the ReAct loop, short for reason-and-act, and it is the engine of autonomy.

The profound shift from series one is this. In ShopHub, *you* drove every turn: the agent acted once per message you sent and then waited for you. Here, after you give the single goal, the agent drives *itself*. It decides to take a second step, a third, a tenth, based on what it keeps discovering, with no human

turning the crank between steps. The agent has stopped being a function you call and become a process that runs.

Picture the difference between asking a librarian a single question and getting a single answer, versus hiring a researcher and leaving for the day. The researcher reads something, realises they need more, pulls another source, notices a contradiction, digs further, and only stops when they judge the work complete. You didn't prompt each of those steps — the researcher's own loop did. That self-driven loop is exactly what we're building.

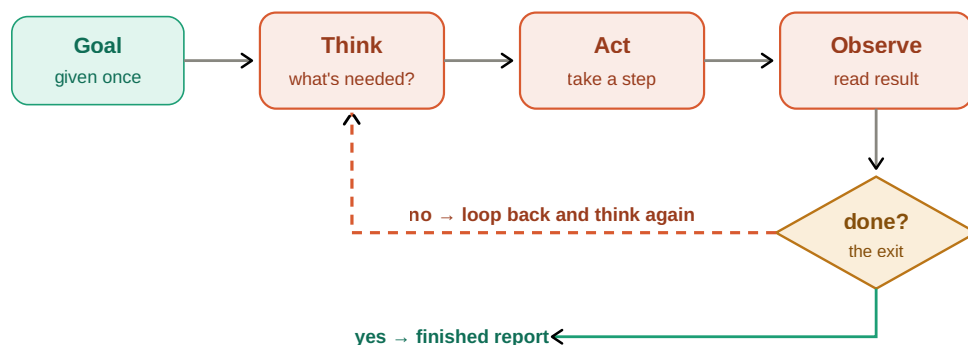
THE ONE IDEA OF THIS LESSON

An autonomous agent runs a loop: think about what's needed, act by taking a step, observe the result, and decide whether to stop or loop again. The human provides the goal once; the loop provides every step after that. This self-driven cycle is what the word "autonomous" actually means in code.

PART 3

The architecture: a loop with an exit

Every diagram in series one flowed in one direction, even when it branched. This one is different in a way you can see at a glance: it has a genuine cycle, an arrow that loops back. The agent thinks, acts, observes, and then a decision either exits with the finished answer or returns to thinking for another round.



The agentic loop. The return arrow is the whole point — it's what lets the agent take another step on its own. The "done?" decision is the exit, without which the loop would run forever.

Two features make this work and distinguish it from everything in series one. First, the loop-back arrow: the agent returns to thinking after each observation, which is how it takes many self-driven steps. Second, the "done?" exit: something must decide when the goal is met, or the agent never stops. Hold both in mind — much of building autonomous agents is getting that exit condition right, a theme we'll return to.

PART 4

Build it in LangGraph: the loop made explicit

Because you met LangGraph in series one, we'll start there, where it has a real strength for this task: LangGraph lets you draw the loop *explicitly* as a graph with a cycle. You can see the loop in the code because you wire it yourself. Here is the agentic loop as a LangGraph graph.

```
from typing import TypedDict, List
from langgraph.graph import StateGraph, END
from langchain_openai import ChatOpenAI
from langchain_core.messages import SystemMessage, HumanMessage

llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)

class ResearchState(TypedDict):
    goal: str          # the single goal the human gave
    findings: List[str] # everything gathered so far
    steps: int          # how many loops we've run (our exit guard)
```

Why this exists: the state carries `findings` that grow each loop and a `steps` counter. That counter is our safety exit — autonomous loops need a guard so they can't run forever.

Now the loop body — a single node that thinks, acts, and observes — and the decision that either loops or exits.

```
def research_step(state: ResearchState) -> ResearchState:
    # THINK + ACT: decide what to look up next and "search" for it.
    prompt = [
        SystemMessage(content="You are researching a goal. Given what you "
                               "already know, state the single most useful next "
                               "fact to find, then provide it."),
        HumanMessage(content=f"Goal: {state['goal']}\nKnown so far: "
                              f"{state['findings']}"),
    ]
    new_finding = llm.invoke(prompt).content # OBSERVE: the result
    state["findings"].append(new_finding)
    state["steps"] += 1
    return state

def should_continue(state: ResearchState) -> str:
```

```
# DECIDE: stop after enough steps, otherwise loop back. This is the exit.
return "stop" if state["steps"] >= 4 else "continue"
```

Why this exists: `research_step` is one turn of think-act-observe. `should_continue` is the "done?" diamond from the diagram — the guard that makes the loop terminate instead of spinning forever.

```
# Wire the cycle. Note the edge from the node BACK to itself via the
# condition — THIS is the loop drawn explicitly in the graph.
graph = StateGraph(ResearchState)
graph.add_node("research", research_step)
graph.set_entry_point("research")
graph.add_conditional_edges("research", should_continue,
                           {"continue": "research", "stop": END})

agent = graph.compile()
```

Why this exists: look at the conditional edge — `"continue"` points back to `"research"` itself. That self-loop is the agentic loop, visible right there in the wiring. In LangGraph, you can always see the cycle.

PART 5

Build it in CrewAI: the loop hidden inside the agent

Now the same idea in CrewAI, and the contrast is illuminating. CrewAI takes a higher-level, more human view: you don't wire the loop yourself. Instead you describe an *agent* by its role, goal, and backstory, give it a *task*, and CrewAI runs the think-act-observe loop for you, internally, until the task is done. The loop is still there — it's just hidden inside the framework rather than drawn by you.

```
from crewai import Agent, Task, Crew

# Describe the agent the way you'd describe a person for a job:
# a role, a goal, and a backstory that shapes how it behaves.
researcher = Agent(
    role="Research Analyst",
    goal="Produce a thorough briefing on the given topic",
    backstory="You are meticulous and never settle for a shallow "
              "first answer. You dig until the picture is complete.",
    verbose=True, # prints each loop step so you can WATCH it think
)
```

Why this exists: notice there is no loop in this code. The agent's drive to "dig until complete" lives in its `backstory` and `goal` — CrewAI turns that description into a loop it runs for you.

```
# A task is the concrete job, with the deliverable spelled out.
briefing = Task(
    description="Research the current state of solid-state batteries.",
    expected_output="A clear briefing covering technology, key players, "
                   "hurdles, and timeline.",
```

```

agent=researcher,
)

# A crew runs the agent on the task. CrewAI executes the internal
# think-act-observe loop until the expected_output is satisfied.
crew = Crew(agents=[researcher], tasks=[briefing])
result = crew.kickoff()

```

Why this exists: `kickoff()` hides the entire loop. CrewAI keeps the agent working — thinking, acting, observing — until the task's `expected_output` is met, all without you writing a cycle.

PART 6

Compare: two philosophies, side by side

You just built the same agentic loop two ways, and the difference between them is not cosmetic — it reflects two genuinely different philosophies about how much the framework should do for you. This is the comparison skill this series is really about, so let's lay it out plainly.

LANGGRAPH VS CREWAI — THE AGENTIC LOOP

	LangGraph	CrewAI
The loop	You wire it explicitly as a graph cycle	Hidden inside the agent; the framework runs it
Control	Total — you see and shape every step and exit	Less — you describe intent, trust the framework
Code amount	More — you build the machinery	Less — a working agent in a few lines
Best when	You need precise, custom, auditable control	You want speed and a natural, role-based model
The risk	More to write and maintain	Less visibility when something goes wrong

Here's the honest takeaway, and it's a lesson that outlives both tools. LangGraph hands you the machinery and asks you to assemble it, which is more work but gives you complete, visible control — you can point at the exact line where the loop decides to continue. CrewAI hands you a finished engine and asks you to describe what you want, which is faster and more intuitive but means the loop's behaviour is shaped by a backstory rather than a line of code you can audit. Neither is better in the abstract. The skill is matching the tool to the job: reach for LangGraph when control and auditability matter most, and for CrewAI when speed and a natural team metaphor matter more.

PART 7

Run it, and the new crack

Run either version on the solid-state battery goal and, unlike the one-shot bot, you can watch it take step after self-driven step.

Goal: Brief me on the current state of solid-state batteries.

[think] I know the basic chemistry. I should find which companies lead.

[act] search "leading solid-state battery companies"

[observe] Found: several firms targeting late-decade production.

[think] Now I need the main manufacturing hurdle.

[act] search "solid-state battery manufacturing challenges"

[observe] Found: scaling the solid electrolyte layer is the key barrier.

Scholar (after 4 loops): A fuller briefing, built step by step from what each search revealed...

This is autonomy you can see. The agent took four steps, each chosen based on what the previous one uncovered, with no prompting from you between them. That self-driven progression is exactly what the one-shot bot could never do. But watch the agent's behaviour closely and a flaw emerges: its steps are *reactive and ad hoc*. It decides each next move in the moment, with no overall plan. Sometimes it wanders into a tangent, sometimes it covers the same ground twice, and it has no sense of whether it's addressing all the parts a good briefing needs.

THE PROBLEM LESSON 2 WILL FIX

Our agent loops, but it improvises every step with no roadmap, so it can wander or repeat itself. In Lesson 2 we'll teach it to *plan first*: before researching, the agent will write its own outline of what the briefing must cover, then work through that plan deliberately. The agent is about to stop wandering and start with a map of its own making.

End of Scholar Lesson 1. You built the agentic loop — the think-act-observe cycle that makes an agent autonomous — in two frameworks, and learned to weigh their trade-offs. Next: giving the agent the ability to plan its own work.